

MauiRAG

Aplicación MAUI para RAG local
con AnythingLLM y LM Studio

Tarea 5 - Sistemas RAG

Programación de Inteligencia Artificial

CE en Inteligencia Artificial y Big Data

Antonio Ferrer Martínez

Febrero 2026

INDICE

PARTE I: DOCUMENTO DE PREPROCESADO DE DATOS

1. Introducción
2. Pipeline de preprocesado general
3. Ejemplo 1: PDF de historial de empleados
4. Ejemplo 2: CSV de empleados
5. Ejemplo 3: Imágenes (Visión + OCR + EXIF)
6. Ejemplo 4: Transcripción de video de YouTube
7. Ejemplo 5: Apuntes en PDF segmentados por página
8. Conclusiones

PARTE II: INFORME DE USO

1. Descripción general
2. Arquitectura MVVM
3. Stack tecnológico
4. Instalación y puesta en marcha
5. Scripts de preprocesado
6. Guía de usuario
7. Notas de compilación

PARTE III: EJEMPLOS DE CASOS DE USO

- Caso 1: Consultas sobre el historial de empleados (PDF)
- Caso 2: Consultas sobre el CSV de salarios
- Caso 3: Consultas sobre imágenes (Visión + OCR + EXIF)
- Caso 4: Video educativo (transcripción de YouTube)
- Caso 5: Apuntes de programación en PDF (segmentados por página)

PARTE I: DOCUMENTO DE PREPROCESADO DE DATOS

1. INTRODUCCIÓN

En este documento explico como he preparado los datos que se usan en el sistema RAG. El objetivo es transformar los documentos originales (PDFs, CSVs, imágenes, etc.) para que AnythingLLM pueda generar los embeddings correctamente y el modelo pueda responder preguntas basándose en esos documentos.

El proceso siempre sigue un patrón similar: primero se obtiene el documento en bruto, se limpia y segmenta en fragmentos (chunks), y finalmente se generan los embeddings que se almacenan en la base de datos vectorial. AnythingLLM se encarga de gran parte de esto de forma automática, pero viene bien saber que pasa por dentro para preparar los datos correctamente.

2. PIPELINE DE PREPROCESADO GENERAL

El flujo que sigue cualquier documento es el siguiente:

```
Documento original --> Extracción de texto --> Segmentación en chunks --> Modelo de embeddings --> Base de datos vectorial
```

Cada paso tiene su importancia:

- Extracción de texto: dependiendo del formato del archivo, la extracción se hace de una forma u otra. No es lo mismo un PDF que un CSV o un texto plano. Si es una imagen, hay que recurrir a OCR y extracción de metadatos.
- Segmentación (chunking): el texto se divide en fragmentos. Si los chunks son demasiado grandes pierden precisión en la búsqueda, y si son demasiado pequeños pierden contexto. AnythingLLM usa por defecto chunks de unos 1000 tokens, que va bastante bien.
- Generación de embeddings: cada chunk pasa por el modelo de embeddings (en mi caso uso el que incluye AnythingLLM, all-MiniLM-L6-v2) y genera un vector numérico. Básicamente, textos parecidos en contenido tendrán vectores parecidos.
- Almacenamiento: los vectores se guardan en la base de datos vectorial (LanceDB, incluida con AnythingLLM). Cuando el usuario hace una pregunta, se buscan los vectores más similares y se pasan esos chunks como contexto al LLM para que elabore la respuesta.

3. EJEMPLO 1: PREPROCESADO DEL PDF DE HISTORIAL DE EMPLEADOS

El primer documento utilizado es el PDF "Historial_Empleados.pdf". Contiene información de 5 empleados: nombre, puesto e historial profesional de cada uno.

3.1 Extracción

Al ser un PDF con texto embebido (no escaneado), AnythingLLM lo procesa directamente. Internamente utiliza un parser de PDF que extrae el texto manteniendo el orden de lectura.

3.2 Segmentación

El documento es breve (1 página), por lo que se genera un solo chunk o a lo sumo 2-3. Cada chunk contiene la información de uno o varios empleados. Al tener una estructura clara con campos Nombre, Puesto e Historial bien diferenciados, los chunks mantienen la coherencia del contenido.

3.3 Resultado

Tras subirlo al workspace y actualizar los embeddings, se pueden hacer preguntas como "¿Cuál es el historial de Juan Pérez?" o "¿Quién tiene más experiencia?" y el sistema encuentra los chunks relevantes y responde con la información del documento.

La ventaja de que el PDF tenga una estructura clara es que los chunks quedan bien delimitados. Si fuera un PDF con tablas complejas o columnas, el resultado sería peor y habría que hacer un preprocesado manual previo.

4. EJEMPLO 2: PREPROCESADO DEL CSV DE EMPLEADOS

El segundo documento es "empleados (1).csv", con datos tabulares: Nombre, Puesto y Salario de los mismos empleados.

4.1 Extracción

Los archivos CSV se procesan línea a línea. AnythingLLM los lee y convierte cada fila en texto plano:

```
"Nombre: Juan Pérez, Puesto: Desarrollador, Salario: 3000"
```

4.2 Consideraciones

Con los CSV hay que tener precaución porque la segmentación puede separar filas relacionadas entre sí. En este caso, al ser un archivo pequeño (5 filas), no supone problema. Sin embargo, con un CSV de miles de registros habría que pensar en agruparlo por algún criterio antes de subirlo.

también me asegure de que el CSV tenía una cabecera con los nombres de las columnas (Nombre, Puesto, Salario), ya que sin ella los chunks no tendrían contexto sobre que representa cada valor.

4.3 Resultado

Al preguntar "¿Cuál es el salario de María Gómez?" el sistema busca en los chunks del CSV y devuelve la información correspondiente. La trazabilidad muestra el documento de origen y el score de similitud.

5. EJEMPLO 3: PREPROCESADO DE IMAGENES (VISION + OCR + EXIF)

Una de las funcionalidades que he implementado en la app es el procesamiento de imágenes (JPG, PNG, etc.) para extraer información e incorporarla al RAG. Como AnythingLLM no acepta imágenes directamente, realizo un preprocesado previo en la propia app que combina tres técnicas: descripción con modelo de visión, OCR y extracción de metadatos EXIF.

5.1 Pipeline de procesamiento

Cuando el usuario selecciona una imagen, la app ejecuta el siguiente flujo:

```
Imagen --> Descripción IA (modelo visión) + Extracción EXIF + OCR + Info archivo  
--> Texto plano (.txt) --> Subida a AnythingLLM --> Embeddings
```

5.2 descripción con modelo de visión

La app envía la imagen codificada en base64 a un modelo multimodal de visión cargado en LM Studio. Utiliza la API compatible con OpenAI de LM Studio (endpoint /v1/chat/completions) con la imagen incluida como image_url en formato data URI.

El modelo analiza la imagen y devuelve una descripción en lenguaje natural de lo que se observa: objetos, personas, colores, escena, etc. Esto es lo que más aporta, porque permite preguntar sobre lo que se ve en la imagen, no solo sobre los metadatos.

Para que funcione es necesario tener cargado un modelo con capacidad de visión en LM Studio, como LLaVA 1.5 7B. La app lo detecta automáticamente consultando el endpoint /v1/models y buscando uno que contenga "llava" o "visión" en el nombre. Si no hay modelo de visión disponible, esa sección se omite y el resto del análisis (EXIF, OCR) sigue funcionando con normalidad.

5.3 Extracción de metadatos EXIF

Utilizo la librería MetadataExtractor (paquete NuGet) para extraer los metadatos EXIF de la imagen:

- Marca y modelo de la cámara
- Fecha de captura
- Parámetros técnicos: exposición, apertura, ISO, focal, flash

- Coordenadas GPS (si las contiene)
- Dimensiones de la imagen
- Software de edición

5.4 OCR (Reconocimiento de texto)

Para el OCR uso la API de Windows.Media.Ocr incluida en Windows. Procesa la imagen con el motor OCR del idioma del sistema y extrae cualquier texto presente: carteles, documentos, capturas de pantalla, etc. Si la imagen no contiene texto, simplemente indica "No se detectó texto".

5.5 Información del archivo

También extraigo información básica del archivo: nombre, tipo, tamaño, fecha de creación y última modificación.

5.6 Generación del texto

Toda la información anterior se reúne en un archivo .txt con secciones diferenciadas:

```
ANALISIS DE IMAGEN: i5092222598.jpg
--- INFORMACION DEL ARCHIVO ---
Nombre: i5092222598.jpg
Tipo: JPG
Tamaño: 18.8 KB
Fecha de creación: 10/07/2025 10:38:01
Ultima modificación: 10/07/2025 10:38:02
--- METADATOS EXIF ---
Marca: Samsung / Modelo: SM-G986B (Galaxy S20+)
(y demás datos EXIF disponibles en la foto)
--- DESCRIPCION DE LA IMAGEN ---
En la imagen se ve un tractor rojo con las palabras "Barreiros"
escritas en el. La escena se desarrolla en frente de una
construcción blanca. Al lado del tractor hay un vehículo antiguo.
En el centro hay un asiento cercano a la parte trasera del tractor.
--- TEXTO DETECTADO (OCR) ---
Barreiros R545S
```

5.7 Resultado

Ese archivo .txt se sube a AnythingLLM como cualquier otro documento y se generan los embeddings. De esta forma se pueden hacer preguntas sobre las imágenes:

- "Describe la imagen que se subió" (usa la descripción del modelo de visión)
- "¿Cuándo se hizo la foto?" (usa los metadatos EXIF)
- "¿Qué texto aparece en la imagen?" (usa el OCR)
- "¿Qué marca es el tractor?" (combina la descripción de visión con el OCR)

La combinación de las tres técnicas (visión + EXIF + OCR) permite que el RAG responda tanto sobre el contenido visual como sobre los datos técnicos de la imagen.

6. EJEMPLO 4: PIPELINE PARA TRANSCRIPCIÓN DE VIDEO DE YOUTUBE

Para este ejemplo he utilizado el video "Programación Orientada a Objetos (POO) Explicada" del canal Fazt en YouTube (<https://www.youtube.com/watch?v=SI7O81GMG2A>). Es un video de 9 minutos y medio que explica los 4 pilares de la POO.

6.1 El problema

AnythingLLM no acepta videos ni URLs de YouTube, solo archivos de texto, así que hay que sacar el contenido del video a texto para meterlo en el RAG.

6.2 De donde se obtiene la transcripción

La mayoría de los videos de YouTube disponen de subtítulos. Estos pueden ser de dos tipos:

- Subtítulos manuales: los añade el creador del video. Suelen tener buena calidad porque están revisados.
- Subtítulos autogenerados: los genera YouTube automáticamente mediante reconocimiento de voz. Funcionan razonablemente bien, aunque pueden contener errores de transcripción.

Estos subtítulos son accesibles a través de la API de YouTube. Existe una librería de Python llamada youtube-transcript-api que facilita su descarga. Lo único que se necesita es el ID del video (la parte que aparece después de "v=" en la URL; en este caso, "SI7O81GMG2A").

El video elegido dispone de subtítulos manuales en español, así que la transcripción sale con buena calidad.

6.3 Herramientas utilizadas

He usado Python con la librería youtube-transcript-api. Se instala con:

```
pip install youtube-transcript-api
```

6.4 Proceso paso a paso

He creado un script en Python (transcribir_video_youtube.py) que realiza todo el proceso. El script es una herramienta general: acepta la URL o el ID de cualquier video de YouTube como argumento de línea de comandos o los pide por consola si se ejecuta sin argumentos. Además, este script está integrado dentro de la propia app MAUI: en la pestaña Documentos hay un campo para pegar la URL de YouTube y un botón que ejecuta el script automáticamente (la app

lo llama con `Process.Start` desde la clase `PythonProcessorService`). Internamente realiza lo siguiente:

1. Extraer el ID del video: A partir de la URL o del ID directo que le pases, el script extrae el identificador de 11 caracteres que YouTube usa internamente. Acepta varios formatos de URL (`youtube.com/watch?v=`, `youtu.be/`, etc.).
2. Descargar los subtítulos del video:

```
from youtube_transcript_api import YouTubeTranscriptApi
api = YouTubeTranscriptApi()
transcript = api.fetch(video_id, languages=['es'])
```

La API devuelve una lista de segmentos. Cada segmento contiene tres campos: el texto del subtítulo (`.text`), el segundo en que comienza (`.start`) y su duración (`.duration`). Por ejemplo, un segmento podría ser: `start=66.44`, `duration=5.2`, `text="Pasemos a la Herencia"`. Si no hay subtítulos en el idioma pedido, intenta descargar los que haya disponibles.

3. Agrupar en bloques de 30 segundos: Los subtítulos individuales son muy breves (2-5 segundos cada uno), lo que resulta insuficiente para que un chunk tenga contexto significativo. El script recorre los segmentos y los va acumulando hasta que la diferencia de tiempo supera los 30 segundos, momento en que comienza un bloque nuevo.
4. Formatear con marcas de tiempo: Cada bloque se escribe con el formato `[Minuto XX:XX - YY:YY]` seguido del texto. El minuto se calcula a partir del campo `.start` de cada segmento. De esta forma, cuando el RAG devuelve un chunk, el usuario puede ver en que minuto del video se trata ese tema.
5. Generar el archivo de salida: Se escribe todo en un archivo `.txt` cuyo nombre se genera automáticamente a partir del ID del video (por ejemplo `transcripción_SI7O81GMG2A.txt`). Incluye una cabecera con la URL del video, la duración y el idioma.

Para ejecutar el script con el video de este caso de uso:

```
python transcribir_video_youtube.py https://www.youtube.com/watch?v=SI7O81GMG2A
```

O de forma interactiva (sin argumentos):

```
python transcribir_video_youtube.py
```

El archivo generado tiene este formato:

```
[Minuto 00:00 - 00:28]
La programación orientada a objetos, POO para los amigos, es un paradigma
de programación que modula nuestro código haciéndolo más entendible y
reutilizable...
[Minuto 06:44 - 07:10]
Pasemos a la, Herencia. Ahora queremos crear la clase Guerrero y Mago, que tienen
```



```
todos los atributos y métodos que tiene nuestra clase Personaje, más algunos atributos y métodos nuevos. En vez de crear todo de nuevo, podemos reutilizar la clase Personaje con la herencia.
```

En total se generan 33 bloques temporales.

6.5 Resultado

Al subir el archivo al workspace "Video POO" y preguntar "¿Qué es la herencia en POO?", el RAG devuelve el chunk con las marcas [Minuto 06:44 - 07:10] y un score de 0.5429. El usuario puede ir directamente a ese minuto del video en YouTube para ver la explicación completa, sin necesidad de ver el video entero.

7. EJEMPLO 5: PIPELINE PARA APUNTES EN PDF (SEGMENTADOS POR PAGINA)

He utilizado el PDF "Aprenda Java como si estuviera en Primero" de TECNUN (Universidad de Navarra), un manual de referencia de 175 páginas sobre Java escrito por J. García de Jalón et al.

7.1 El problema

Cuando se sube un PDF directamente a AnythingLLM, el sistema extrae el texto y lo divide en chunks, pero se pierde la referencia a la página de origen. Es decir, si el RAG devuelve un chunk con información sobre constructores, no sabes de qué página viene. Si quieres ir a la página original del PDF a consultar algo, no puedes.

7.2 Herramientas utilizadas

He usado Python con la librería PyPDF2. Se instala con:

```
pip install PyPDF2
```

7.3 Proceso paso a paso

He creado un script en Python (segmentar_pdf_por_pagina.py) que realiza todo el proceso. El script es una herramienta general: acepta la ruta de cualquier PDF como argumento de línea de comandos o la pide por consola si se ejecuta sin argumentos. Opcionalmente se le puede indicar un rango de páginas para procesar solo una parte del PDF. Además, este script está integrado dentro de la app MAUI: al seleccionar un PDF en la pestaña Documentos aparece un checkbox "Procesar PDF con marcas [Pagina XX]" que, al marcarlo, ejecuta el script automáticamente antes de subir el documento (via Process.Start desde PythonProcessorService). Si no se marca el checkbox, el PDF se sube directamente a AnythingLLM sin preprocesar. Internamente realiza lo siguiente:

1. Abrir el PDF con PyPDF2:

```
from PyPDF2 import PdfReader
reader = PdfReader(pdf_path)
# reader.pages es una lista con todas las páginas del PDF
```

2. Determinar el rango de páginas: Si se indica un rango (por ejemplo 37-64), solo se procesan esas páginas. Si no se indica, se procesan todas. Para este caso de uso seleccione las páginas 37-64, que corresponden al capítulo 3 "Clases en Java". Estas 28 páginas cubren los conceptos de POO: clases, objetos, constructores, herencia, polimorfismo, interfaces y clases abstractas.

3. Extraer el texto página a página:

```
text = reader.pages[idx].extract_text()
```

4. Limpiar cabeceras repetidas: Muchos PDFs académicos incluyen en cada página una cabecera con el copyright y el nombre del capítulo. El script las elimina mediante expresiones regulares (`re.sub`) para que no interfieran en los chunks.

5. Anadir [Pagina XX] al inicio de cada línea: Este es el paso más importante. Se recorren los párrafos de cada página y se les añade la marca de página al principio:

```
parrafos = [p.strip() for p in text.split('\n') if p.strip()]
for parrafo in parrafos:
    lineas.append(f"[Pagina {num_pagina}] {parrafo}")
```

6. Generar el archivo de salida: Se escribe todo en un archivo .txt cuyo nombre se genera automáticamente a partir del nombre del PDF (por ejemplo `aprenda_java_primer_segmentado.txt`). Incluye una cabecera con el nombre del PDF y el rango de páginas procesadas.

Para ejecutar el script con el PDF de este caso de uso:

```
python segmentar_pdf_por_pagina.py aprenda_java_primer.pdf 37-64
```

O de forma interactiva (sin argumentos):

```
python segmentar_pdf_por_pagina.py
```

El archivo generado tiene este formato:

```
[Pagina 37] 3. CLASES EN JAVA
[Pagina 37] Las clases son el centro de la Programación Orientada a Objetos (OOP).
[Pagina 37] Algunos de los conceptos más importantes de la POO son los siguientes:
[Pagina 44] Un constructor es un método que se llama automáticamente cada vez que se crea
[Pagina 44] un objeto de una clase. La principal misión del constructor es reservar memoria
[Pagina 44] e inicializar las variables miembro de la clase.
```

7.4 ¿Por qué [Pagina XX] en cada línea y no sólo al inicio de cada página?

Inicialmente coloque [Pagina XX] solo al principio de cada página, en una línea separada. Al probarlo, descubrí que no funcionaba correctamente: AnythingLLM al trocear el texto separaba la marca del contenido, y el chunk que devolvía el RAG no incluía la referencia a la página. La solución fue cambiar el formato para que cada línea individual lleve la marca. De esta forma, independientemente de donde AnythingLLM realice el corte, la página siempre aparece en el chunk.

7.5 Resultado

Al subir el archivo al workspace "Apuntes Java" y preguntar "¿Qué es un constructor en Java?", el RAG devuelve chunks con las marcas [Pagina 50] y [Pagina 44] claramente visibles en el texto, con scores de 0.5997 y 0.6241. El alumno puede ir directamente a la página 44 del PDF original para consultar la sección completa sobre constructores.

8. CONCLUSIONES

El preprocesado importa bastante en un sistema RAG. Si los documentos están mal preparados, las respuestas van a ser malas. Lo que he sacado en claro de esta práctica:

- El tamaño de los chunks es clave: ni demasiado grandes ni demasiado pequeños. Los 1000 tokens por defecto de AnythingLLM van bastante bien.
- Meter metadatos (página, minuto, sección) dentro de los chunks ayuda mucho a saber de dónde viene cada cosa.
- Los documentos bien estructurados (con cabeceras y secciones claras) dan mejores resultados que los bloques de texto sin formato.
- Las imágenes necesitan un preprocesado especial (visión + OCR + EXIF) para convertirlas a texto que se pueda indexar.
- Los videos de YouTube hay que descargarles los subtítulos y formatearlos con marcas de tiempo.
- Los PDFs extensos se benefician de una segmentación por página con marcas [Pagina XX] en cada línea, no solo al inicio.
- AnythingLLM automatiza casi todo, pero cuando el formato del documento es complicado hay que entender cómo funciona por dentro para preparar bien los datos.
- Haber integrado los scripts de Python dentro de la app (con Process.Start) hace que todo sea más cómodo: el usuario no tiene que abrir una terminal para preprocesar nada, lo hace todo desde la interfaz.

PARTE II: INFORME DE USO

1. DESCRIPCIÓN GENERAL

MauiRAG es una aplicación multiplataforma hecha con .NET MAUI que se conecta con AnythingLLM para poder hacer consultas RAG desde una interfaz gráfica. La app permite gestionar workspaces, subir documentos (incluyendo imágenes), configurar el system prompt y hacer preguntas en modo chat o query, viendo de dónde vienen las respuestas (trazabilidad).

La aplicación sigue el patrón MVVM (Model-View-ViewModel) y se organiza en 4 pestañas principales con iconos:

- Chat (icono de mensaje): pantalla principal para interactuar con el RAG
- Workspaces (icono de carpeta): crear, renombrar y eliminar workspaces
- Documentos (icono de documento): subir archivos y asignarlos a un workspace
- Config (icono de engranaje): configurar la URL, API key y system prompt

2. ARQUITECTURA MVVM

La app usa el patrón MVVM para separar la lógica de la interfaz:

Estructura de carpetas:

- Modelos/ -> clases de datos (WorkspaceItem, DocumentoItem, MensajeChat, ChunkInfo)
- ViewModels/ -> lógica de cada pantalla (ChatViewModel, WorkspacesViewModel, DocumentosViewModel, ConfigViewModel, BaseViewModel)
- Pages/ -> las vistas XAML de cada pantalla
- Servicios/ -> ApiService (llamadas a la API), ImageProcessorService (procesamiento de imágenes) y PythonProcessorService (preprocesado de PDFs y videos de YouTube)

Cada pantalla tiene su ViewModel con INotifyPropertyChanged y ObservableCollection, así la interfaz se actualiza sola cuando cambian los datos. Los comandos (ICommand) enlazan los botones con los métodos del ViewModel.

BaseViewModel implementa INotifyPropertyChanged y tiene un método SetProperty para no repetir el código de notificación en cada ViewModel.

3. STACK TECNOLÓGICO

Tecnologías y herramientas utilizadas en el proyecto:

Aplicación:

- .NET MAUI (net10.0-windows10.0.19041.0): framework para la interfaz gráfica multiplataforma

- C#: lenguaje de programación de la app
- Patron MVVM: arquitectura de la aplicación
- Newtonsoft.Json 13.0.3: serialización/deserialización de JSON para la comunicación con la API
- MetadataExtractor 2.8.1: extracción de metadatos EXIF de imágenes
- Windows.Media.Ocr: reconocimiento de texto en imágenes (OCR nativo de Windows)

Infraestructura RAG:

- LM Studio (<https://lmstudio.ai>): servidor local de modelos de lenguaje. Expone una API compatible con OpenAI en localhost:1234
- Llama 3.2 1B Instruct: modelo de texto para generar respuestas (ligero, funciona con GPU)
- LLaVA 1.5 7B: modelo de visión para describir imágenes (opcional, funciona en CPU)
- AnythingLLM Desktop (<https://anythingllm.com>): plataforma RAG que gestiona documentos, embeddings y workspaces. Expone una API REST en localhost:3001
- all-MiniLM-L6-v2: modelo de embeddings incluido en AnythingLLM (embeddings nativos)
- LanceDB: base de datos vectorial incluida en AnythingLLM

Scripts de preprocesado (integrados en la app):

- Python 3: lenguaje para los scripts de preprocesado de documentos (se ejecutan automáticamente desde la app via Process.Start)
- youtube-transcript-api: librería para descargar subtítulos de YouTube
- PyPDF2: librería para extraer texto de archivos PDF

Nota sobre la elección de modelos:

Para el modelo de texto probe varios modelos disponibles en LM Studio antes de elegir Llama 3.2 1B Instruct. También probe Qwen 2.5 1.5B Instruct y Phi-3.1 Mini 4K, pero Llama 3.2 1B dio el mejor equilibrio entre velocidad de respuesta y calidad para las consultas RAG en mi hardware (GPU con 4GB VRAM). Los modelos más grandes (7B+) daban mejor calidad pero eran demasiado lentos para una experiencia de uso fluida.

Para los embeddings use all-MiniLM-L6-v2, qué es el que viene incluido con AnythingLLM como "embedder nativo" y no necesita configuración adicional. AnythingLLM también permite usar modelos de embeddings externos (como nomic-embed-text a través de Ollama o modelos de OpenAI), pero el nativo va bien para documentos en español y además no necesita ningún servicio extra.

Para el modelo de visión (procesamiento de imágenes) use LLaVA 1.5 7B, qué es de los pocos modelos multimodales que se pueden ejecutar en local con LM Studio. Al ser un modelo de 7B parámetros lo cargo en CPU, lo cual lo hace más lento, pero funciona sin necesitar VRAM adicional.

4. INSTALACIÓN Y PUESTA EN MARCHA

4.1 Instalar LM Studio

1. Descargar LM Studio desde <https://lmstudio.ai> (disponible para Windows, Mac y Linux)
2. Instalar y abrir la aplicación
3. Desde la pestaña de búsqueda, descargar los modelos:
 - Llama 3.2 1B Instruct (para texto): buscar "llama-3.2-1b-instruct" y descargar la versión GGUF
 - LLaVA 1.5 7B (para visión, opcional): buscar "llava-v1.5-7b" y descargar
4. Ir a la pestaña "Local Server" (icono de servidor en el lateral)
5. Cargar el modelo de texto (Llama 3.2 1B Instruct)
6. Pulsar "Start Server". El servidor arranca en <http://localhost:1234>
7. Si se quiere usar procesamiento de imágenes, cargar también el modelo de vision en una segunda instancia o slot

4.2 Instalar AnythingLLM Desktop

1. Descargar AnythingLLM Desktop desde <https://anythingllm.com> (sección Download)
2. Instalar y abrir la aplicación
3. En la configuración inicial:
 - LLM Provider: seleccionar "LM Studio" e introducir la URL <http://localhost:1234/v1>
 - Embedding Provider: seleccionar "AnythingLLM Embedder" (usa all-MiniLM-L6-v2, no necesita configuración adicional)
 - Vector Database: dejar LanceDB (viene por defecto)
4. Generar una API Key desde la configuración de AnythingLLM (apartado API Keys). Esta clave se necesita para que MauiRAG pueda conectarse.
5. AnythingLLM queda corriendo en <http://localhost:3001>

4.3 Compilar y ejecutar MauiRAG

1. Abrir la solución MauiRAG.csproj en Visual Studio
2. Seleccionar el target framework: net10.0-windows10.0.19041.0

3. Compilar con F5 o Ctrl+Shift+B
4. Al abrir la app, ir a la pestaña Config e introducir la URL (<http://localhost:3001/api>) y la API Key generada en el paso anterior
5. Pulsar "Probar Conexión" para verificar que conecta correctamente

4.4 Orden de arranque

El orden para arrancar todo el sistema es:

1. Primero: LM Studio (cargar modelos y arrancar el servidor local)
2. Segundo: AnythingLLM Desktop (se conecta a LM Studio para el LLM y embeddings)
3. Tercero: MauiRAG (se conecta a AnythingLLM para gestionar workspaces y hacer consultas)

Los scripts de preprocesado (casos 4 y 5) están integrados en la app y se ejecutan automáticamente cuando se sube un PDF o se introduce una URL de YouTube. Para que funcionen es necesario:

- Python 3 instalado y en el PATH del sistema
- Librerías: `pip install youtube-transcript-api PyPDF2`
- Configurar la ruta de los scripts y del ejecutable de Python en la pestaña Config de la app

5. SCRIPTS DE PREPROCESADO (Casos 4 y 5)

Para los casos de uso 4 y 5 he creado dos scripts de Python independientes que preparan los documentos antes de subirlos a AnythingLLM. Los scripts son herramientas generales reutilizables: aceptan cualquier video de YouTube o cualquier PDF, no están atados a un archivo concreto. Se encuentran en la carpeta raíz del proyecto, junto a la solución de MAUI.

Además, los scripts están integrados dentro de la propia app MAUI: cuando el usuario sube un PDF o pega una URL de YouTube en la pestaña Documentos, la app llama automáticamente al script correspondiente mediante `Process.Start` (clase `PythonProcessorService`). Así el usuario no necesita abrir una terminal para ejecutarlos, todo se hace desde la interfaz gráfica. Para los PDFs hay un checkbox que permite elegir si se quiere procesar el PDF con marcas de página o subirlo directamente tal cual.

5.1 transcribir_video_youtube.py

Este script descarga los subtítulos de cualquier video de YouTube y genera un archivo de texto con marcas de tiempo [Minuto XX:XX - YY:YY]. Acepta la URL del video o su ID como argumento de línea de comandos, o si se ejecuta sin argumentos los pide por consola.

YouTube permite que los creadores de contenido añadan subtítulos a sus videos, y también genera subtítulos automáticos mediante reconocimiento de voz. Estos subtítulos se pueden descargar a través de la API de YouTube. También se podría copiar la transcripción a mano desde la propia interfaz de YouTube (en el desplegable del video aparece la opción "Mostrar transcripción"), pero hacerlo manualmente no sería la forma adecuada: se perderían las marcas de tiempo exactas, habría que formatear todo el texto a mano y para videos largos sería inviable.

El script automatiza todo el proceso: descarga los subtítulos con sus timestamps, los agrupa en bloques de 30 segundos y genera el archivo listo para subir. El nombre del archivo de salida se genera automáticamente a partir del ID del video (por ejemplo transcripción_SI7O81GMG2A.txt).

Requisitos e instalación:

```
pip install youtube-transcript-api
```

Ejecución (con argumentos):

```
python transcribir_video_youtube.py https://www.youtube.com/watch?v=SI7O81GMG2A
python transcribir_video_youtube.py SI7O81GMG2A es
```

Ejecución (interactiva, sin argumentos):

```
python transcribir_video_youtube.py
```

El segundo argumento es opcional e indica el idioma de los subtítulos (por defecto 'es'). Si no hay subtítulos en ese idioma, el script intenta descargar los que haya disponibles.

Para el caso de uso 4 se uso con el video "Programación Orientada a Objetos (POO) Explicada" de Fazt (ID: SI7O81GMG2A). El archivo generado (transcripción_SI7O81GMG2A.txt) se renombro a transcripción_poo_video.txt y se subió al workspace "Video POO" desde la pestaña Documentos de la app.

Para más detalles sobre el proceso técnico, ver la sección 6 del Documento de Preprocesado.

5.2 segmentar_pdf_por_pagina.py

Este script extrae el texto de cualquier PDF página a página y genera un archivo de texto con marcas [Pagina XX] al inicio de cada línea. Acepta la ruta del PDF como argumento de línea de comandos, o si se ejecuta sin argumentos la pide por consola. Opcionalmente se puede indicar un rango de páginas para procesar solo una parte del PDF.

Esto es necesario porque AnythingLLM, al procesar un PDF directamente, pierde la referencia a la página de origen en los chunks. Con este formato, cualquier chunk que genere AnythingLLM siempre contiene la página de donde viene el texto.

El nombre del archivo de salida se genera automáticamente a partir del nombre del PDF (por ejemplo manual_segmentado.txt).

Requisitos e instalación:

```
pip install PyPDF2
```

Ejecución (con argumentos):

```
python segmentar_pdf_por_pagina.py manual.pdf  
python segmentar_pdf_por_pagina.py aprenda_java_primer.pdf 37-64
```

Ejecución (interactiva, sin argumentos):

```
python segmentar_pdf_por_pagina.py
```

El segundo argumento es opcional e indica el rango de páginas a procesar (por ejemplo 10-50).

Si no se indica, se procesan todas las páginas del PDF.

Para el caso de uso 5 se uso con el PDF "Aprenda Java como si estuviera en Primero" de TECNUN (Universidad de Navarra), que se puede descargar desde:

```
https://www.tecnun.es/jgjalon/Asignaturas/JAVA/ApsJava.pdf
```

Se proceso el rango de páginas 37-64 (capítulo 3: Clases en Java). El archivo generado (aprenda_java_primer_segmentado.txt) se renombro a apuntes_java_clases.txt y se subió al workspace "Apuntes Java" desde la pestaña Documentos de la app.

Para más detalles sobre el proceso técnico y por que se pone [Pagina XX] en cada línea, ver la sección 7 del Documento de Preprocesado.

6. GUIA DE USUARIO

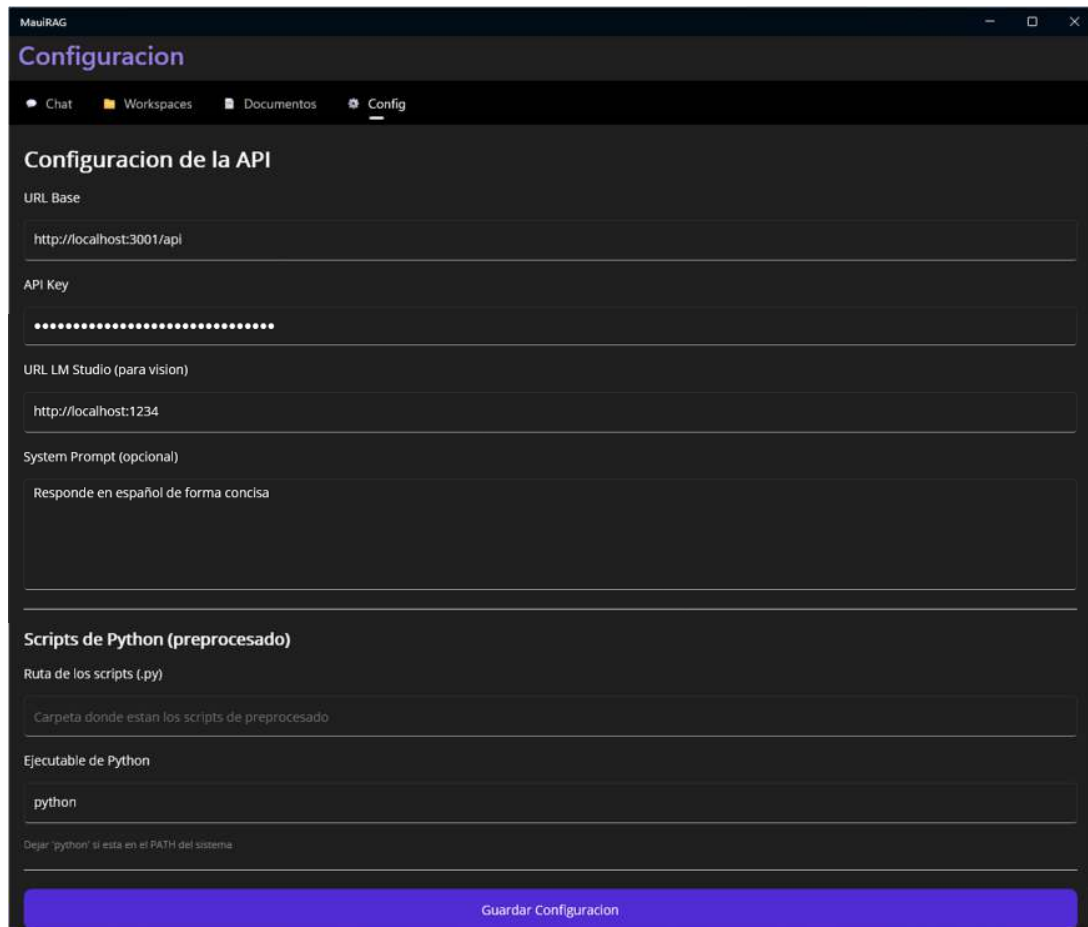
6.1 PANTALLA DE CONFIGURACIÓN (pestaña "Config")

Es lo primero que hay que hacer al abrir la app por primera vez. Aqui se configuran varias cosas:

- URL Base: la dirección donde está corriendo AnythingLLM. Por defecto es `http://localhost:3001/api`. Si lo tienes en otro puerto o en otra máquina hay que cambiarlo aquí.
- API Key: la clave de la API de AnythingLLM. Se genera desde la configuración de AnythingLLM, en el apartado de API keys. Sin esto la app no puede conectarse.
- System Prompt: es opcional. Aquí puedes escribir un prompt de sistema que se envía con cada consulta. Por ejemplo "Responde siempre en español y de forma concisa". Se guarda automáticamente para que no lo pierdas al cerrar la app.
- Ruta de los scripts (.py): la carpeta donde están los scripts de preprocesado (segmentar_pdf_por_pagina.py y transcribir_video_youtube.py). Es necesario para que la app pueda ejecutarlos automáticamente al subir PDFs o videos de YouTube.
- Ejecutable de Python: la ruta al ejecutable de Python (por defecto "python"). Si Python está en el PATH del sistema se puede dejar el valor por defecto.

Al pulsar "Guardar Configuración" los valores se guardan en las preferencias del dispositivo y se aplican inmediatamente.

También hay un botón "Probar Conexión" que comprueba si la app puede conectarse a AnythingLLM con la configuración actual. Muestra un mensaje verde si la conexión es correcta o rojo si falla. Es útil para verificar que la URL y la API key son correctas antes de empezar a trabajar.



pantalla Config con los campos rellenos y el botón de probar conexión

6.2 PANTALLA DE WORKSPACES (pestaña "Workspaces")

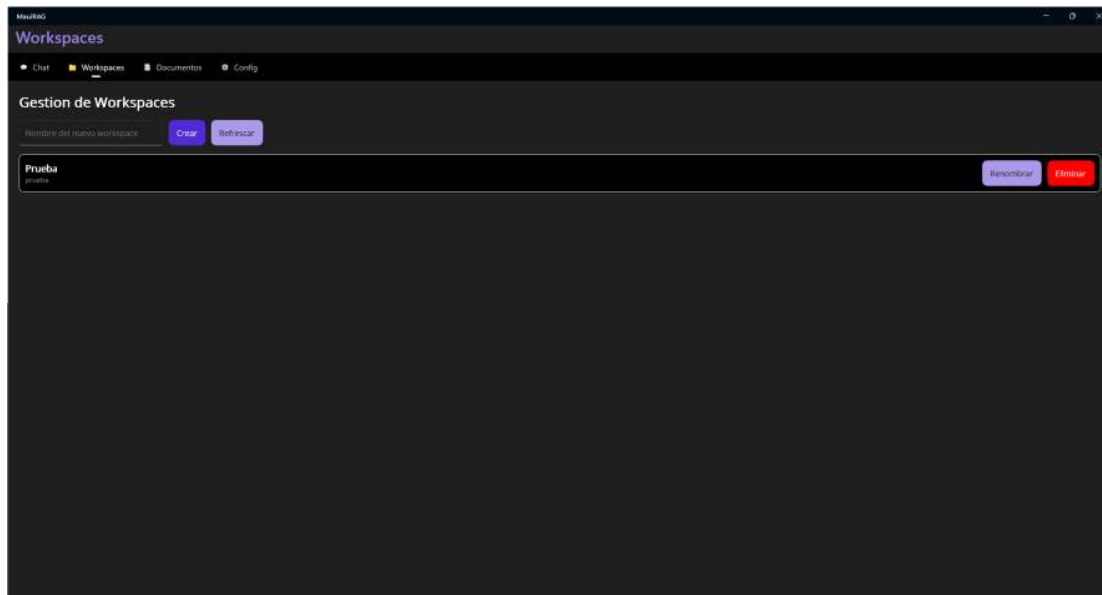
Aquí se gestionan los workspaces de AnythingLLM. La pantalla muestra:

- Un campo de texto y botón "Crear" arriba para crear workspaces nuevos
- Un botón "Refrescar" para recargar la lista
- La lista de workspaces existentes, cada uno con su nombre, slug, y botones de "Renombrar" y "Eliminar"

Para crear un workspace: escribes el nombre y pulsas "Crear". Te saldrá un aviso de que se ha creado correctamente.

Para renombrar: pulsas "Renombrar" en el workspace que quieras y te sale un popup para escribir el nuevo nombre.

Para eliminar: pulsas "Eliminar" (botón rojo) y te pide confirmación antes de borrarlo. Esto elimina el workspace y todos sus documentos asociados, así que hay que tener cuidado.



pantalla Workspaces con la lista de workspaces y el formulario de crear

6.3 PANTALLA DE DOCUMENTOS (pestaña "Documentos")

Esta pantalla tiene tres secciones:

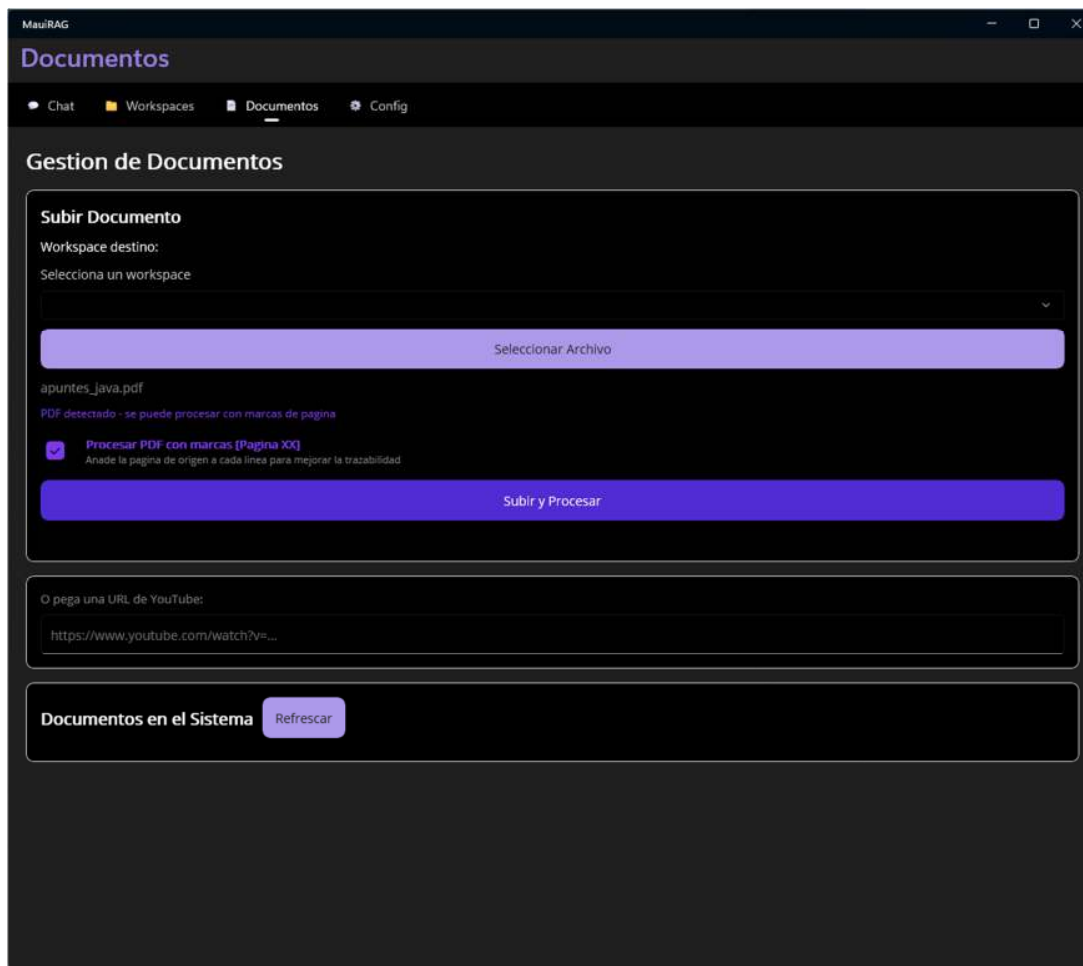
Sección superior - Subir Documento:

- Picker para elegir el workspace destino
- Botón "Seleccionar Archivo" para abrir el explorador de archivos
- Acepta los siguientes formatos: PDF, TXT, CSV, DOCX, MD y también imágenes (JPG, JPEG, PNG, BMP, GIF, TIFF, WEBP)
- Una vez seleccionado el archivo aparece su nombre y tipo debajo (por ejemplo "Documento de texto", "PDF detectado" o "Imagen detectada")
- Si el archivo seleccionado es un PDF, aparece un checkbox "Procesar PDF con marcas [Pagina XX]" que permite elegir si se quiere preprocesar el PDF con el script de Python (añadiendo la página de origen a cada línea) o subirlo directamente tal cual a AnythingLLM

- Botón "Subir y Procesar" que hace la subida y actualización de embeddings

Si el archivo es una imagen, la app hace un preprocesado especial antes de subirla: pide a un modelo de visión (cargado en LM Studio) que describa lo que se ve en la imagen, extrae los metadatos EXIF (cámara, fecha, GPS, etc.) y hace OCR para detectar texto. Todo se junta en un archivo .txt que se sube a AnythingLLM para generar los embeddings. La app detecta automáticamente el modelo de visión disponible en LM Studio (busca uno con "llava" o "visión" en el nombre), así que no hace falta configurar el nombre del modelo. Si no hay modelo de visión cargado, el resto del análisis (EXIF + OCR) sigue funcionando normal.

Si el archivo es un PDF y el checkbox está marcado, la app llama al script `segmentar_pdf_por_pagina.py` que extrae el texto del PDF y añade [Pagina XX] al inicio de cada línea. Si el checkbox no está marcado, el PDF se sube directamente a AnythingLLM sin preprocesar.



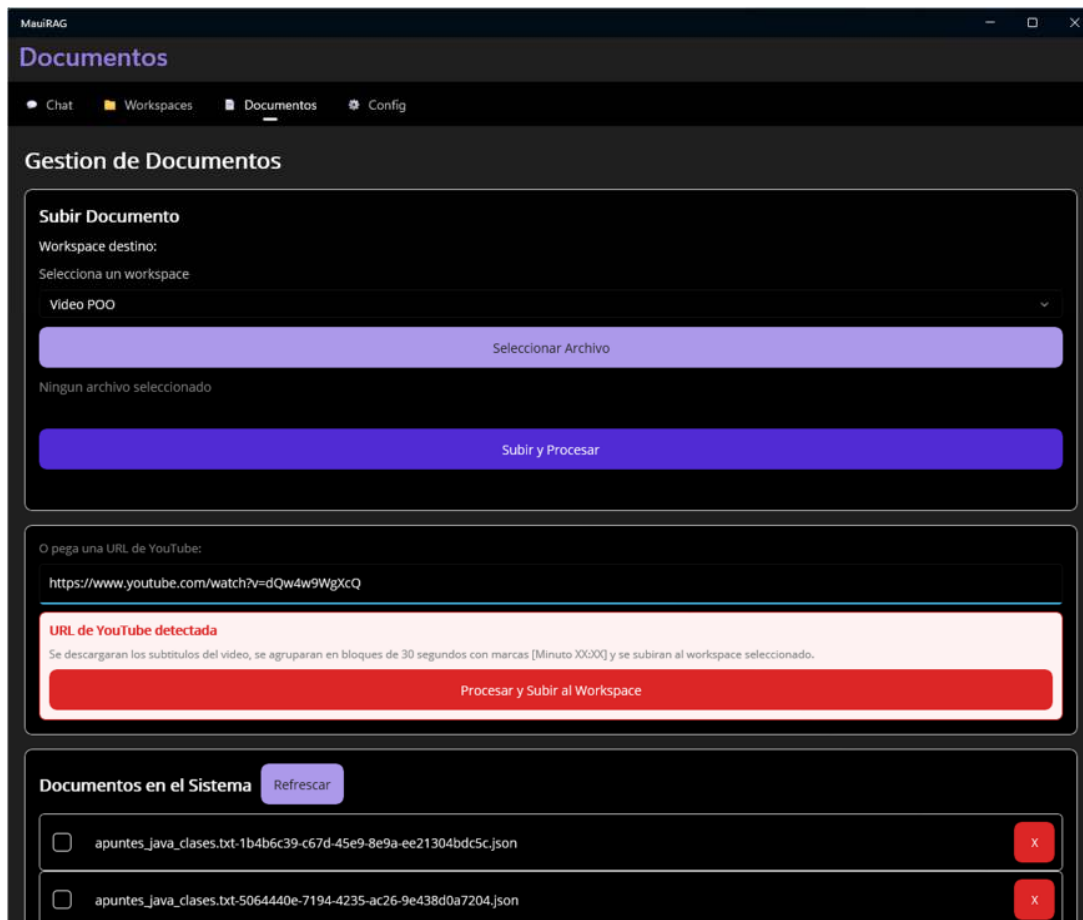
pantalla Documentos con un PDF seleccionado y el checkbox de marcas de página visible

El proceso de subida muestra mensajes de estado:

- "Procesando imagen (Visión IA + OCR + EXIF)..." (si es una imagen)
- "Procesando PDF con marcas [Pagina XX]..." (si es un PDF con el checkbox marcado)
- "Subiendo documento..." (en naranja)
- "Documento 'X' subido. Actualizando embeddings..."
- "Documento 'X' subido y embeddings actualizados!" (en verde si todo fue bien)
- Mensajes de error en rojo si algo falla

Sección media - URL de YouTube:

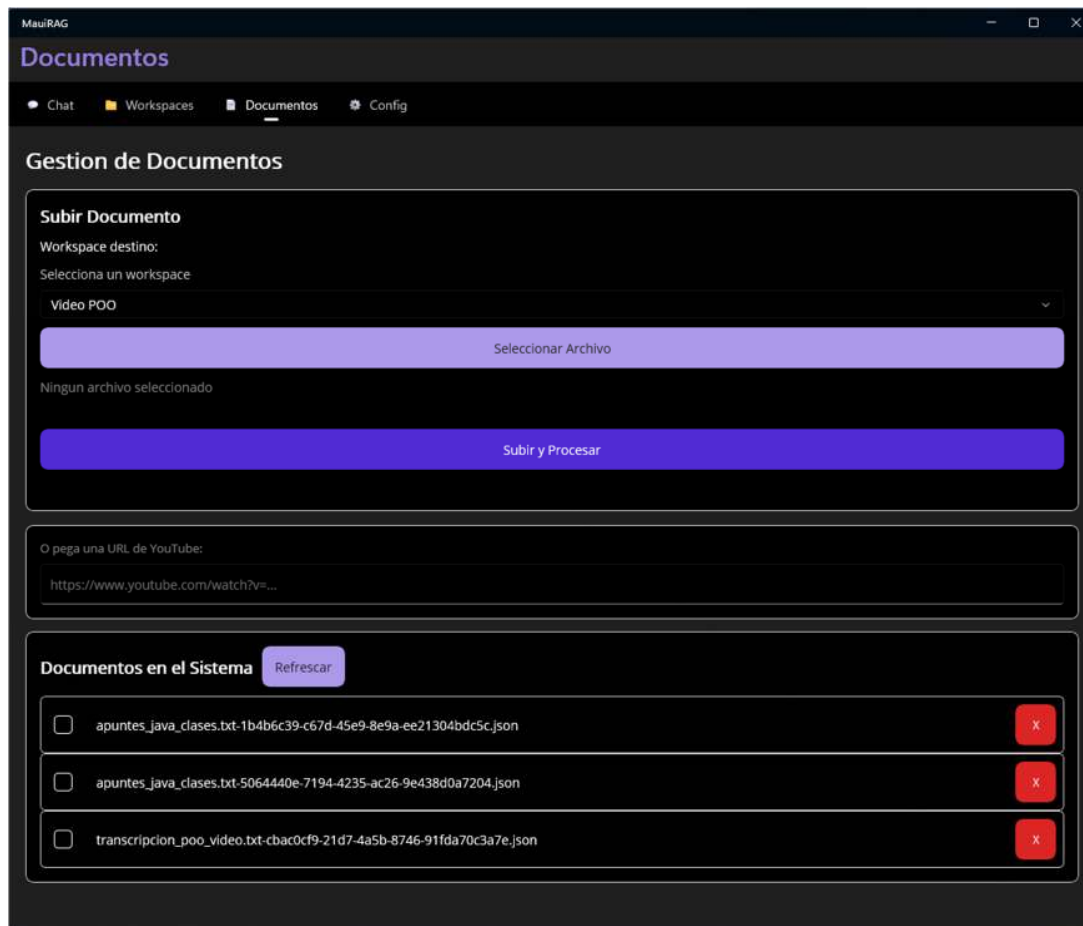
- Un campo de texto discreto donde se puede pegar la URL de un video de YouTube
- Cuando la app detecta que la URL es de YouTube (válida el formato), aparece automáticamente un panel informativo en rojo que dice "URL de YouTube detectada" y explica que se van a descargar los subtítulos, agruparlos en bloques de 30 segundos con marcas [Minuto XX:XX] y subirlos al workspace
- El botón "Procesar y Subir al Workspace" solo aparece cuando hay una URL válida detectada
- Todo el proceso es automático: la app extrae el ID del video, ejecuta el script de Python y sube el resultado



pantalla Documentos con URL de YouTube detectada y panel rojo visible

Sección inferior - Documentos en el Sistema:

- Lista de todos los documentos que hay en AnythingLLM
- Cada documento tiene un checkbox para seleccionarlo
- Botón "Refrescar" para actualizar la lista
- Botón "Eliminar Seleccionados" (en rojo) para borrar los documentos marcados, con confirmación previa



pantalla Documentos con un archivo seleccionado y el picker de workspace

6.4 PANTALLA DE CHAT (pestaña "Chat")

Esta es la pantalla principal donde se interactúa con el RAG. Tiene estas partes:

Parte superior:

- Picker para seleccionar el workspace sobre el que quieres preguntar
- Botón "Limpiar Chat" para borrar la conversación y empezar de cero

Selector de modo:

- Chat: mantiene el historial de conversación. Las preguntas anteriores influyen en las respuestas siguientes. Es más conversacional.
- Query: cada pregunta es independiente, no mantiene contexto. Es más preciso para consultas puntuales.

Area de chat:

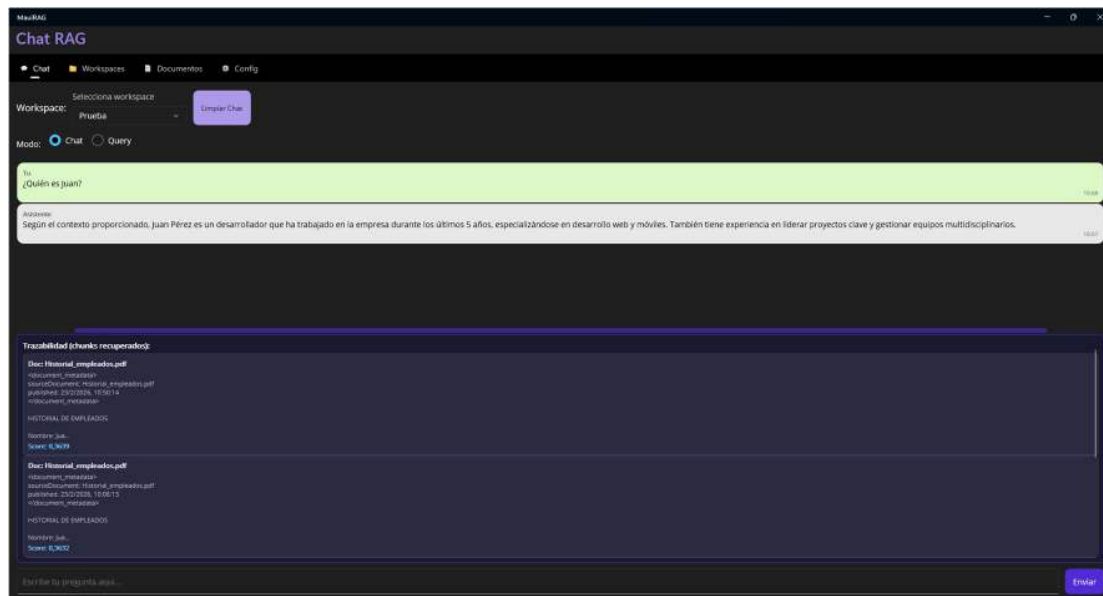
- Los mensajes del usuario aparecen en verde a la derecha con la hora
- Las respuestas del asistente aparecen en gris a la izquierda con la hora
- Se hace scroll automatico al último mensaje
- Se usa un CollectionView con DataTemplate para mostrar los mensajes (patrón MVVM)

Sección de trazabilidad:

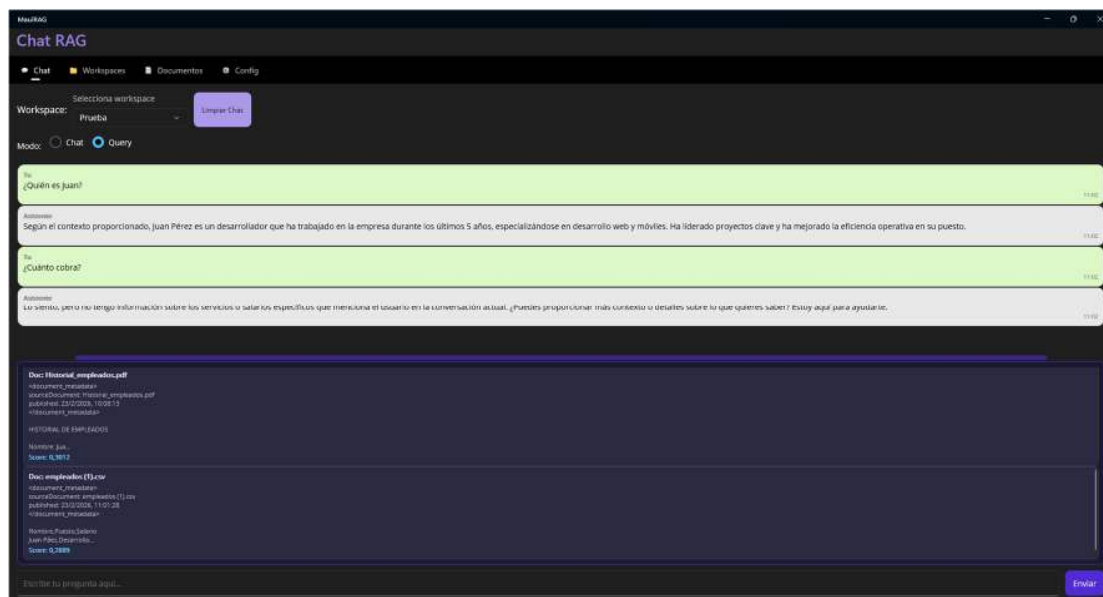
- Debajo del chat hay un recuadro con tema oscuro que muestra los chunks que el RAG ha recuperado
- El panel es redimensionable: se puede arrastrar el handle de la parte superior para hacerlo más grande o más pequeño (entre 80 y 500 pixeles)
- Tiene scroll por si los chunks no caben
- Para cada chunk se muestra:
 - El nombre del documento de origen (Doc: nombre_archivo)
 - Un extracto del texto del chunk (recortado a 150 caracteres)
 - El score de similitud (un número entre 0 y 1, cuanto más alto más relevante)
- Se puede tocar un chunk para expandirlo y ver el texto completo, y tocarlo otra vez para contraerlo
- Al expandir un chunk cambia el color del borde (a azul) para indicar qué esta expandido

Campo de entrada:

- Abajo hay un campo de texto para escribir la pregunta y un botón "Enviar"
- Mientras se procesa se muestra un indicador de carga y el botón se desactiva
- También se puede enviar pulsando Enter en el campo de texto



pantalla Chat en modo Chat con una conversación y los chunks de trazabilidad



pantalla Chat en modo Query mostrando una consulta puntual con trazabilidad

7. NOTAS DE COMPILACIÓN

El proyecto está hecho con .NET MAUI targeteando net10.0. Para compilarlo:

1. Abrir la solución MauiRAG.csproj en Visual Studio
2. Seleccionar el target framework de Windows (net10.0-windows10.0.19041.0)
3. Compilar con F5 o Ctrl+Shift+B

Dependencias:

- Microsoft.Maui.Controls
- Microsoft.Maui.Essentials
- Newtonsoft.Json 13.0.3
- MetadataExtractor 2.8.1 (para leer metadatos EXIF de imágenes)

Para el OCR se usa Windows.Media.Ocr que viene con Windows, no necesita paquete adicional. Esta funcionalidad solo está disponible en Windows (compilación condicional con `#if WINDOWS`).

El proyecto compila sin errores. Los warnings que salen son por nullable references y DisplayAlert en .NET 10, pero funciona sin problemas.

Para el preprocesado de PDFs y videos de YouTube desde la app, es necesario tener Python 3 instalado con las librerías youtube-transcript-api y PyPDF2, y configurar la ruta de los scripts en la pestaña Config.

PARTE III: EJEMPLOS DE CASOS DE USO

CASO 1: CONSULTAS SOBRE EL HISTORIAL DE EMPLEADOS (PDF)

Contexto:

Se ha subido el documento "Historial_empleados.pdf" al workspace. Este PDF contiene información de 5 empleados con sus nombres, puestos y trayectoria profesional.

Ejemplo de uso en modo Chat:

Usuario: ¿Quién es el empleado con más experiencia?

Asistente: Según la información del documento, el empleado con más experiencia es Luis Ramírez,

que cuenta con 10 años de experiencia como Gerente de Proyecto. Ha gestionado equipos multidisciplinarios y ha asegurado la entrega de proyectos dentro de los plazos propuestos.

Trazabilidad:

- Doc: Historial_empleados.pdf
- Texto: "Luis tiene una sólida experiencia de 10 años en la gestión de proyectos tecnológicos..."
- Score: 0.8234

Usuario: ¿Y en que se ha especializado?

Asistente: Luis se ha especializado en la gestión de proyectos tecnológicos. En concreto, ha gestionado equipos multidisciplinarios y se ha encargado de asegurar que los proyectos se entreguen dentro de los plazos y presupuestos estipulados.

(En modo chat mantiene el contexto de qué estamos hablando de Luis)

Ejemplo en modo Query:

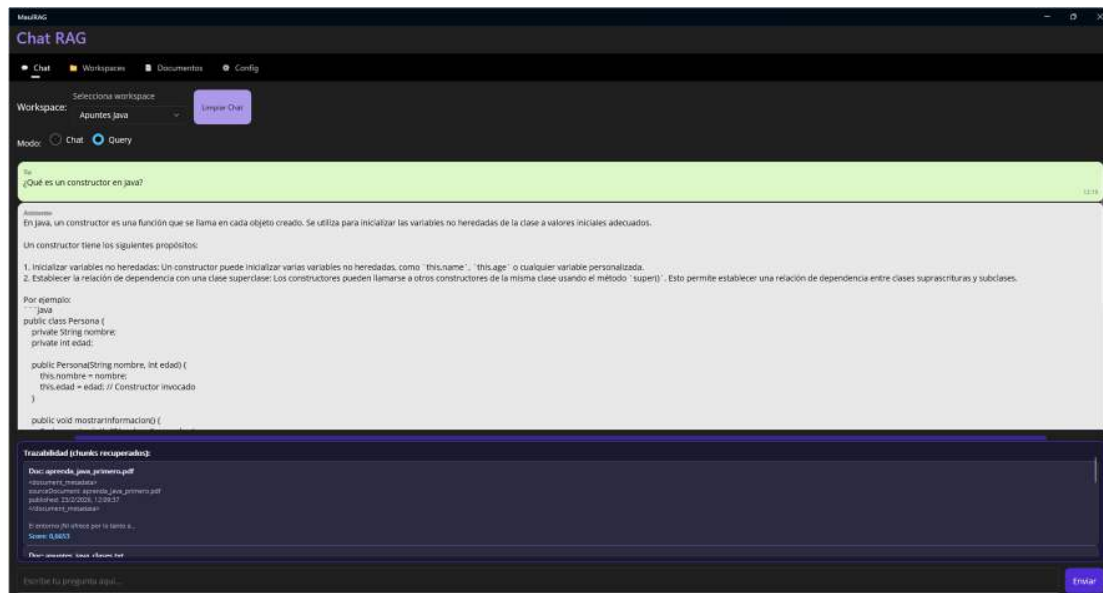
Usuario: ¿Que puesto tiene María Gómez?

Asistente: María Gómez ocupa el puesto de Diseñadora.

Trazabilidad:

- Doc: Historial_empleados.pdf
- Texto: "Nombre: María Gómez. Puesto: Diseñadora. Historial: María cuenta con 7 años..."
- Score: 0.9102

Lo interesante aquí es que en la trazabilidad se ve claramente de que documento viene la información y que trozo de texto ha usado el modelo para responder. Además se puede tocar el chunk para expandirlo y ver el texto completo.



chat PDF con trazabilidad

CASO 2: CONSULTAS SOBRE EL CSV DE SALARIOS

Contexto:

Se ha subido el archivo "empleados (1).csv" a otro workspace. Contiene datos tabulares con nombre, puesto y salario.

Ejemplo:

Usuario: ¿Cuál es el salario más alto?

Asistente: El salario más alto corresponde a Luis Ramírez, que como Gerente de Proyecto cobra 4500 euros.

Trazabilidad:

- Doc: empleados (1).csv
- Texto: "Luis Ramírez, Gerente de Proyecto,4500"
- Score: 0.7856

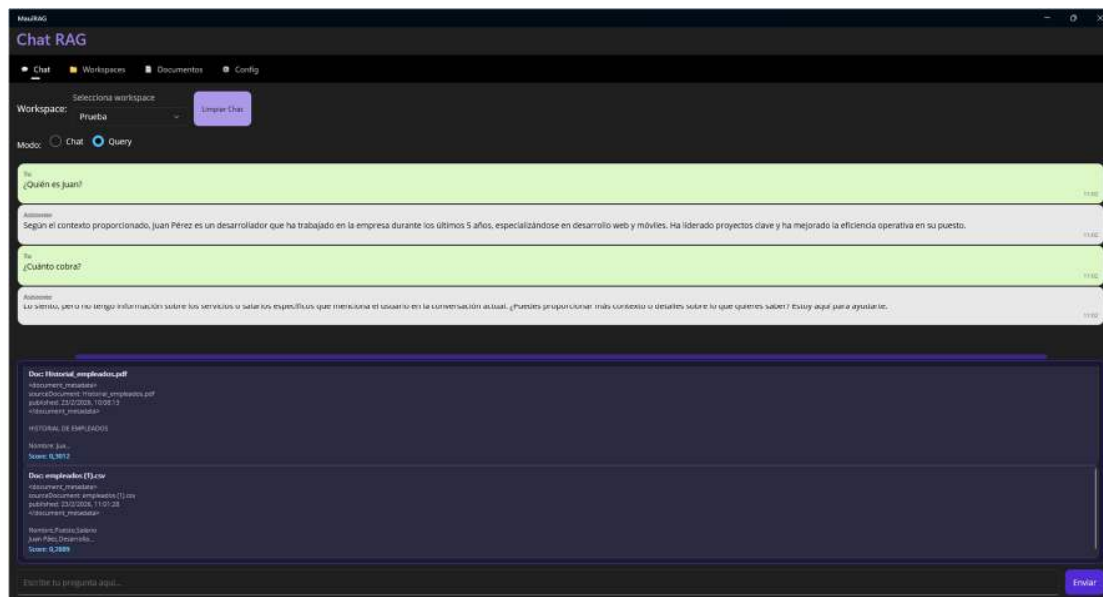
Usuario: ¿Cuánto cobra el desarrollador?

Asistente: El desarrollador Juan Páez tiene un salario de 3000 euros.

Trazabilidad:

- Doc: empleados (1).csv
- Texto: "Juan Pérez, Desarrollador,3000"
- Score: 0.8521

En este caso vemos como el RAG busca en los datos del CSV y encuentra las filas relevantes. El chunk que devuelve es literalmente la línea del CSV que contiene la respuesta.



chat CSV con trazabilidad

CASO 3: CONSULTAS SOBRE IMAGENES (VISION + OCR + EXIF)

Contexto:

Se ha subido una imagen (por ejemplo, una foto hecha con el móvil) al workspace. La app automáticamente procesa la imagen de tres formas: pide a un modelo de visión (LLaVA en LM Studio) que describa lo que se ve, extrae los metadatos EXIF y hace OCR para detectar texto. Todo se junta en un .txt que se indexa en el RAG.

Ejemplo preguntando por el contenido visual:

Usuario: Describe la imagen que se subió

Asistente: Según el contexto, la imagen parece ser una vista de un tractor rojo con las palabras "Barreiros" escritas en él. La escena se desarrolla en frente de una construcción blanca, y al lado del tractor hay un vehículo antiguo. En el centro de la imagen hay un

asiento cercano a la parte trasera del tractor.

Trazabilidad:

- Doc: i5092222598_analisis.txt
- Texto: "--- DESCRIPCIÓN DE LA IMAGEN --- ...un tractor rojo con las palabras Barreiros..."
- Score: 0.4194

Ejemplo preguntando por datos técnicos:

Usuario: ¿Cuándo se hizo la foto?

Asistente: La foto fue tomada el 10 de julio de 2025 a las 10:38.

Trazabilidad:

- Doc: i5092222598_analisis.txt
- Texto: "Fecha de creación: 10/07/2025 10:38:01..."
- Score: 0.3856

Ejemplo preguntando por texto en la imagen (OCR):

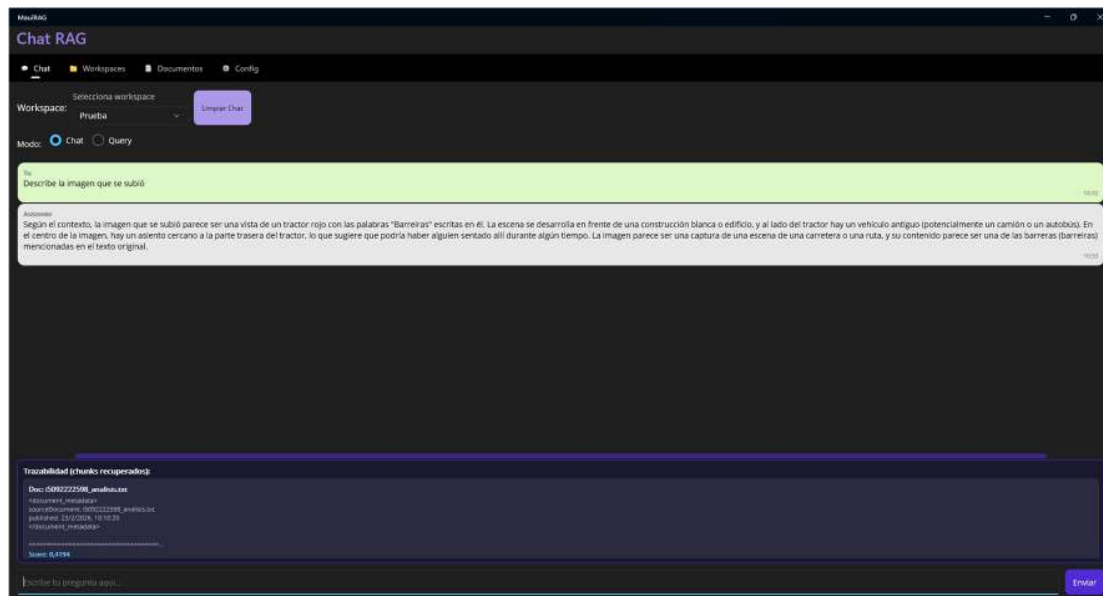
Usuario: ¿Qué texto aparece en la imagen?

Asistente: En la imagen se detecta el texto "Barreiros" y "R545S", que corresponden a la marca y modelo del tractor.

Trazabilidad:

- Doc: i5092222598_analisis.txt
- Texto: "--- TEXTO DETECTADO (OCR) --- Barreiros R545S..."
- Score: 0.9503

Este caso es interesante porque AnythingLLM no acepta imágenes directamente. La app las preprocesa con tres técnicas: descripción con modelo de visión (para entender el contenido visual), metadatos EXIF (fecha, etc.) y OCR (texto detectado). Con las tres cosas juntas, el RAG puede contestar preguntas sobre lo que se ve en la foto y también sobre los datos técnicos.



chat imagen con trazabilidad

CASO 4: VIDEO EDUCATIVO (TRANSCRIPCIÓN DE YOUTUBE)

Contexto:

He usado el video "Programación Orientada a Objetos (POO) Explicada" del canal Fazt en YouTube (<https://www.youtube.com/watch?v=SI7O81GMG2A>). Es un video de 9 minutos y medio que explica los 4 pilares de la POO: abstracción, encapsulamiento, herencia y polimorfismo.

Para obtener la transcripción he pegado la URL del video en la sección "Transcripción de YouTube" de la pestaña Documentos de la app. Al pulsar "Procesar y Subir", la app llama automáticamente al script `transcribir_video_youtube.py` (con `Process.Start` desde `PythonProcessorService`), que descarga los subtítulos con marcas de tiempo, los agrupa en bloques de ~30 segundos y los formatea con marcas [Minuto XX:XX - YY:YY]. El resultado se sube directamente al workspace "Video POO" sin tener que hacer nada más.

Ejemplo en modo Query:

Usuario: ¿Qué es la herencia en POO?

Asistente: En el lenguaje de programación Object-Oriented (POO), la herencia se refiere a un patrón de diseño donde una clase (la clase padre) puede inherir las características y los métodos de otra clase (la clase hija). La clase padre utiliza al menos algunos de los atributos y métodos de la clase hija. (...)

Trazabilidad (primer chunk expandido):

- Doc: transcripción_poo_video.txt
- Texto: "...[Minuto 06:37 - 06:44] Definir atributos y métodos como públicos o privados

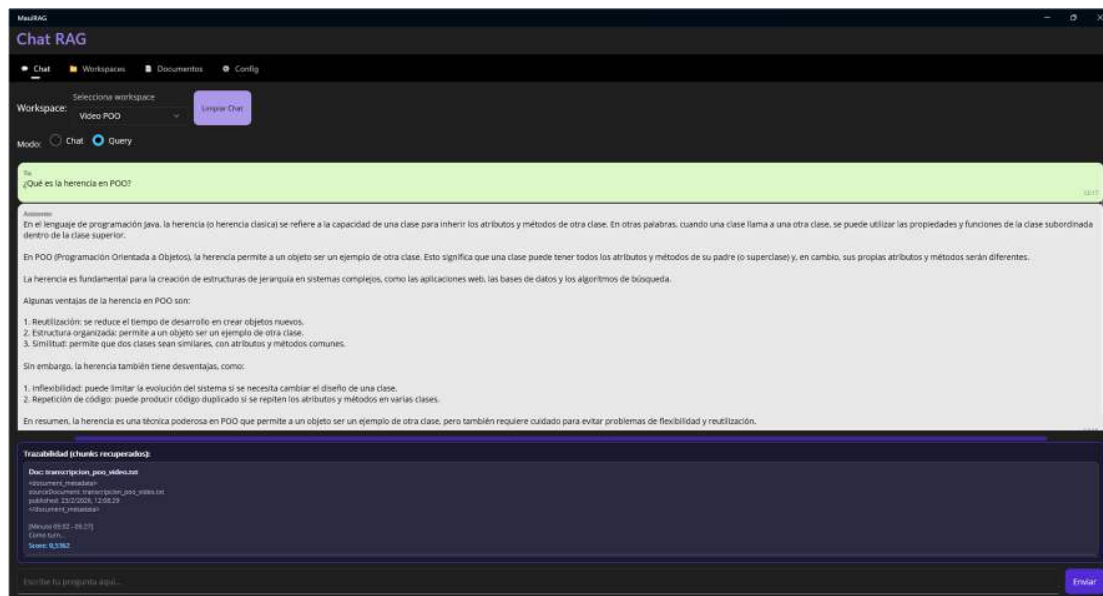
depende del lenguaje, [Minuto 06:44 - 07:10] eso lo veremos en otro video...
Pasemos a la,
Herencia. Ahora queremos crear la clase Guerrero y Mago, que tienen todos los
atributos y
métodos que tiene nuestra clase Personaje... podemos reutilizar la clase
Personaje con la
herencia. [Minuto 07:11 - 07:19]"

- Score: 0.5429

Segundo chunk:

- Texto: "[Minuto 05:02 - 05:27] Como turno es quien controla la ejecución de los métodos..."
- Score: 0.5281

El usuario ve que la información viene de los minutos 06:44-07:10 del video, exactamente donde se explica la herencia. Puede ir directamente a ese punto en YouTube para ver la explicación en formato video. Así un estudiante puede repasar un tema puntual sin tragarse todo el video.



chat video POO con trazabilidad

CASO 5: APUNTES DE PROGRAMACIÓN EN PDF (SEGMENTADOS POR PAGINA)

Contexto:

He usado el PDF "Aprenda Java como si estuviera en Primero" de TECNUN (Universidad de Navarra), un clásico de 175 páginas escrito por J. García de Jalón et al. He extraído el capítulo 3 "Clases en Java" (páginas 37-64) que cubre los conceptos de POO: clases, objetos, constructores, herencia, polimorfismo, interfaces y clases abstractas.

Como AnythingLLM no conserva la referencia a la página de origen al trocear un PDF, he seleccionado el PDF en la pestaña Documentos de la app y he marcado el checkbox "Procesar PDF con marcas [Pagina XX]". Al pulsar "Subir y Procesar", la app llama automáticamente al script `segmentar_pdf_por_pagina.py` (con `Process.Start` desde `PythonProcessorService`), que extrae el texto página a página usando PyPDF2 y añade [Pagina XX] al inicio de cada línea. También limpia las cabeceras repetidas (copyright, nombre del capítulo). El resultado se sube directamente al workspace "Apuntes Java".

Ejemplo en modo Query:

Usuario: ¿Qué es un constructor en Java?

Asistente: En Java, el Constructor es un método que se llama automáticamente cada vez que se crea una nueva instancia de una clase. Su propósito principal es inicializar las variables miembro de la clase cuando se crea un nuevo objeto. Un Constructor tiene las siguientes características:

- 1. No tiene valor de retorno (ni siquiera void)*
- 2. Nombre coincide con el de la clase*
- 3. El argumento implícito es el objeto que se está creando (...)*

Trazabilidad (chunks expandidos):

- Doc: `apuntes_java_clases.txt`
- Texto: "[Pagina 50] constructor previamente definido en la misma clase. En este contexto,

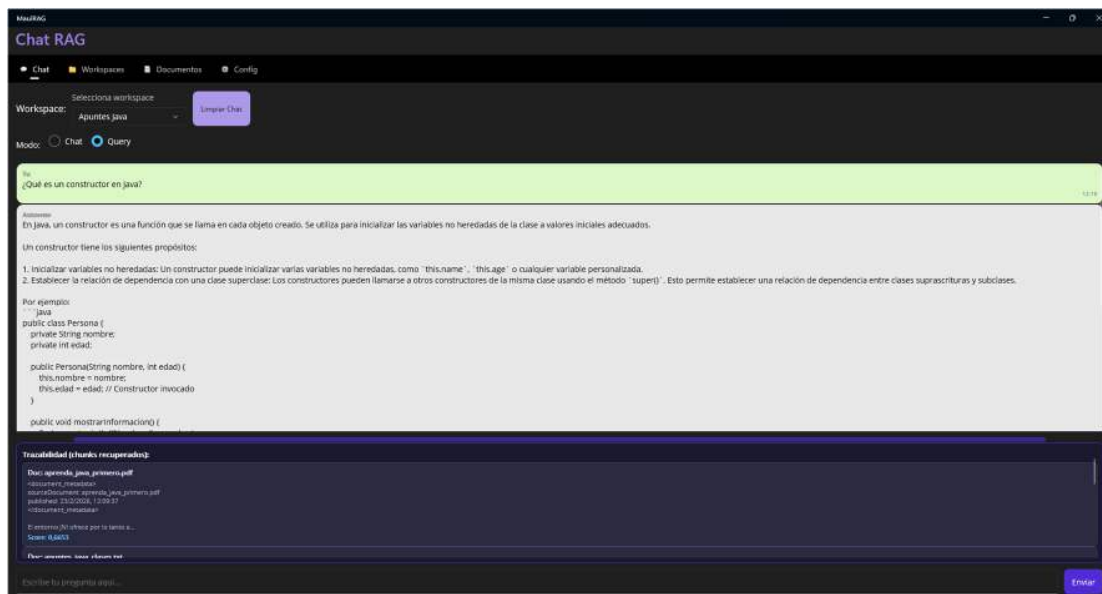
```
la palabra this solo puede [Pagina 50] aparecer en la primera sentencia de un
constructor.
[Pagina 50] De forma analoga el constructor de una clase derivada puede llamar al
constructor
de su [Pagina 50] super-clase por medio de la palabra super()..."
```

- Score: 0.5997
- Doc: apuntes_java_clases.txt
- Texto: "[Pagina 44] uso de constructores. [Pagina 44] Un constructor es un método que se

llama automáticamente cada vez que se crea un objeto de [Pagina 44] una clase. La principal misión del constructor es reservar memoria e inicializar las variables [Pagina 44] miembro de la clase. [Pagina 44] Los constructores no tienen valor de retorno (ni siquiera void) y su nombre es el mismo que [Pagina 44] el de la clase..."

- Score: 0.6241

Se ve que el RAG devuelve chunks con las marcas [Pagina 50] y [Pagina 44] directamente en el texto. Como el preprocesado mete [Pagina XX] en cada línea, el alumno sabe que tiene que ir a la página 44 del PDF. Sin ese preprocesado, no sabría de qué página viene el chunk.



chat apuntes Java con trazabilidad

RESUMEN

Los 5 casos muestran las capacidades de la aplicación con datos reales:

1. Modo Chat con historial: permite hacer preguntas de seguimiento sin repetir contexto (caso 1)
2. Modo Query sin contexto: para consultas puntuales y directas (casos 2-5)
3. Trazabilidad: siempre se muestra de donde viene la información (documento, texto, score), con chunks expandibles para ver el texto completo
4. Flexibilidad de documentos: funciona con PDFs, CSVs, TXT y cualquier formato que acepte AnythingLLM
5. Procesamiento de imágenes: la app preprocesa imágenes (descripción con IA + OCR + EXIF) para poder indexarlas y hacer consultas tanto sobre el contenido visual como sobre los datos técnicos (caso 3)
6. Transcripción de video: las marcas de tiempo [Minuto XX:XX] permiten redirigir al usuario al punto exacto del video donde se habla del tema (caso 4)
7. Apuntes segmentados por página: las marcas [Pagina XX] permiten que el alumno vaya directamente a la página del PDF donde está la información (caso 5)

Lo de la trazabilidad es clave: el usuario puede comprobar de donde sale cada respuesta y no tiene que fiarse a ciegas del modelo. Los chunks se pueden expandir pulsando sobre ellos para ver el texto completo si el extracto recortado no es suficiente.